

AI 데이터 분석 · 3일차

자료형과 연산자

값의 종류를 알고, 계산하고, 참·거짓으로 판단하기

int · float · str · bool · 산술 · 비교 · 논리 연산

종류를 알고(자료형) → 계산하고(산술) → 견주어 판단한다(비교·논리)
그 판단(True/False)이 다음 시간 "조건문"의 재료가 됩니다

2026-07-15 AI 데이터 분석 교육 정리 (3일차)

학습자 Minho · 국립금오공과대학교 컴퓨터공학부

이 책을 읽는 법

어제 배운 변수(값에 이름 붙이기)에서 한 걸음 더 들어갑니다. 오늘의 두 강의 교안(02_02 자료형 · 02_03 연산자)을 하나로 엮었습니다. 파란 세로줄은 파이썬 코드, 초록 ▶ 줄은 실행 결과입니다. 뱃지로 "이게 자료형인지 · 연산자인지 · 내장함수인지"를 먼저 구분한 뒤 정의 · 예제 · 왜 쓰나 순서로 읽으면 됩니다.

자료형 int·float·str·bool **내장함수** type() **연산자** 산술 **비교** ==<> **논리** and·or·not

목차

	장 / STEP	무엇을 배우나
0장	오늘의 출발점 — 값의 "종류"를 보다	변수에서 자료형으로 · type()의 등장
STEP 1	네 가지 기본 자료형	int 정수 · float 실수 · str 문자열 · bool 불린
STEP 2	type()으로 확인 · 자료형의 함정	동작 차이 · 실수 오차 · 문자열 속 숫자
STEP 3	산술 연산 완성하기	거듭제곱 ** · 몫// 나머지% · 우선순위 · += · 문자열
STEP 4	비교 연산자 — 판단의 시작	== != < > <= >= → 결과는 bool
STEP 5	논리 연산자 — 조건 엮기	and · or · not · 조합과 우선순위
부록 A	자료형 · 연산자 치트시트	한눈에 보는 총정리표
부록 B	오류 · 함정 사전	true · TypeError · 실수 오차 · 대소문자
부록 C	설비 실습 하이라이트 & 다음 시간	불량률 · 가동률 · 정비 판정 / 조건문 · 입력 예고

CHAPTER 0

오늘의 출발점 — 값의 "종류"를 보다

변수에서 자료형으로, 그리고 판단(True/False)까지

어제 우리는 값에 이름을 붙이는 "변수"를 배웠습니다. 오늘 04_variable.py 맨 아래에 딱 한 줄이 추가됐죠 — `print(type(temp))`. 바로 이 한 줄이 오늘의 문을 엽니다. 값에 이름을 붙였으면, 이제 그 값이 "어떤 종류(자료형)"인지 볼 차례입니다.

개념

파이썬에서 모든 값은 "객체"이고, 값의 종류가 바로 **자료형(data type)**입니다.

종류에 따라 할 수 있는 일이 다릅니다. 숫자는 계산되고(3+5=8), 글자는 이어붙입니다("3"+"5"="35").

그래서 데이터를 다루려면 종류부터 알아야 합니다.

오늘의 큰 그림은 세 걸음입니다. ①값의 종류를 알고(자료형) → ②그 값으로 계산하고(산술 연산) → ③값을 건주어 참·거짓으로 판단한다(비교·논리 연산). 이 마지막 판단이 만들어내는 True/False가, 다음 시간에 배울 "조건문(if)"의 재료가 됩니다. 오늘은 그 재료를 만드는 날입니다.

오늘 나오는 것	무엇인가	예시
자료형	값의 네 가지 기본 종류	3 25.5 "센서A" True
내장함수	값의 종류를 알려주는 도구	type(25) → int
연산자	계산하는 산술 기호	+ - * / // % **
비교	두 값을 건주어 참·거짓	== != < > <= >=
논리	참·거짓을 엮는 기호	and or not

STEP 1

네 가지 기본 자료형

int · float · str · bool — 딱 네 개만 확실히

자료형은 종류가 많아 보이지만, 오늘은 기본 네 가지만 확실히 잡으면 됩니다. 각 대표 예시 하나씩만 외워두세요 — 정수 3, 실수 25.5, 문자열 "센서A", 불린 True.

자료형 int (정수) 소수점 없는 수

정의 소수점이 없는 수. 개수·나이·횟수처럼 "딱 떨어지는" 값에 씁니다.

```
count = 3      # 센서 개수
age = 27       # 나이
zero = 0
```

왜 쓰나 설비에서 "센서 3개", "점검 12회"처럼 셀 수 있는 값이 정수입니다. 계산이 가능한 진짜 숫자예요.

자료형 float (실수) 소수점 있는 수

정의 소수점이 있는 수. 온도·평균·비율처럼 "정밀한" 값에 씁니다. 정수 5와 실수 5.0은 서로 다른 종류입니다.

```
temp = 25.5      # 온도
height = 173.5
a = 5            # int
b = 5.0          # float (같은 5라도 종류가 다름)
print(b)         # 5.0
```

▶ 5.0

왜 쓰나 설비 측정값(온도 25.5°C, 진동 2.3)은 대부분 실수입니다. 나눗셈 결과도 실수라서, 평균·비율 계산에서 자연히 등장합니다.

자료형 `str` (문자열) 따옴표로 감싼 글자

정의 따옴표로 감싼 글자. 이름·상태·라벨처럼 "글자 그대로" 다룰 값에 씁니다.

```
name = "센서A"
empty = ""      # 빈 문자열도 str
a = 3           # 숫자
b = "3"         # 글자 (따옴표!)
print("3" + "5") # 35 (계산 아님, 이어붙임)
```

▶ 35

왜 쓰나 항목 이름("온도"), 설비 이름("펌프A"), 상태("정상") 같은 글자가 문자열입니다. 따옴표가 있으면 무조건 문자열이에요.

주의

따옴표 규칙: 큰따옴표("")·작은따옴표('') 둘 다 되지만 여는 것과 닫는 것의 종류는 같아야 합니다. 글자 안에 따옴표가 있으면 바깥은 다른 종류로 감쌉니다.

자료형 `bool` (불린) 참·거짓 두 가지

정의 참(True)과 거짓(False) 두 값만 가지는 자료형. 켜짐/꺼짐, 정상/비정상 같은 "두 상태"에 씁니다.

```
ok = True
no = False
# a = true      # x 오류! true는 없는 이름 (참은 True)
b = "True"      # 글자 str (주의 — 불린 아님)
```

왜 쓰나 설비가 "정상인가?"처럼 예/아니오로 답하는 값입니다. 뒤에서 배울 비교 연산의 결과가 바로 이 불린이라, 판단의 핵심이 됩니다.

자주 하는 실수

첫 글자를 대문자로 — True/False가 정답입니다. true(소문자)는 "그런 이름 없음"이라 NameError. 따옴표를 씌운 "True"는 불린이 아니라 그냥 글자예요.

개념 자료형 한눈에 구분하기 3초 판별

정의 값만 보고 종류를 가르는 순서: 따옴표 → 소수점 → True/False.

```
"100"    → 따옴표 있음    → str
100.0    → 따옴표 X, 소수점 0 → float
100      → 따옴표 X, 소수점 X → int
True     → True/False      → bool
```

왜 쓰나 어제 "함수·메서드·속성"을 3초 판별법으로 나눴듯, 자료형도 이 순서면 즉시 구분됩니다. 헛갈리면 다음 STEP의 type()으로 확인하면 끝.

STEP 2

type()으로 확인 · 자료형의 함정

종류가 의심되면 추측 말고 확인하라

내장함수 type() 값의 종류를 알려줌

정의 괄호 안 값의 자료형을 알려주는 내장 함수. 결과는 <class 'int'> 같은 형태로 나옵니다.

형태 type(값)

```
print(type(25))      # <class 'int'>
print(type(25.5))    # <class 'float'>
print(type("센서A")) # <class 'str'>
print(type(True))    # <class 'bool'>
```

```
▶ <class 'int'>
▶ <class 'float'>
▶ <class 'str'>
▶ <class 'bool'>
```

왜 쓰나 계산이 이상할 때 "왜 이러지?" 추측하지 말고 type()으로 종류부터 봅니다. print(값)으로 값을 보고, type(값)으로 종류를 보는 2단계 점검이 습관이 되어야 해요.

개념

확인 몇 초, 안 해서 해매는 오류는 몇 분입니다. 특히 어딘가에서 "받아 온 값"이나 이상한 계산 결과일 때 `type()` 이 가장 강력합니다.

개념 자료형마다 동작이 다르다 같은 + 인데 결과가 다름

정의 같은 + 기호라도 숫자면 계산, 문자열이면 이어붙이기입니다.

```
print(3 + 5)          # 8      (숫자 계산)
print("3" + "5")      # 35     (글자 이어붙임)
print("안녕" + "하세요") # 안녕하세요
```

▶ 8
▶ 35
▶ 안녕하세요

왜 쓰나 "3+5가 왜 35가 나오지?"의 답은 대부분 "사실 문자열이었다"입니다. 종류가 동작을 결정한다는 걸 보여주는 대표 예입니다.

개념 실수의 소수점 오차 $0.1 + 0.2 \neq 0.3$?!

정의 실수 계산은 아주 작은 오차가 생길 수 있습니다. 컴퓨터가 소수를 이진법으로 저장하는데, 0.1처럼 딱 떨어지지 않는 수가 있기 때문입니다.

```
print(0.1 + 0.2)
# 0.30000000000000004
```

▶ 0.30000000000000004

왜 쓰나 버그가 아니라 컴퓨터의 원리입니다. 값이 "정확히 같은지" 비교해야 할 땐 `round(값, 자릿수)`로 자릿수를 맞춘 뒤 비교하세요.

주의

어제 배운 `round()`가 여기서 쓰입니다: `round(0.1 + 0.2, 1) → 0.3`. 돈·측정값을 다룰 때 특히 조심.

개념 문자열 속 숫자 vs 진짜 숫자 "123"은 계산 못 함

정의 겉보기엔 같은 123이라도, 따옴표가 있으면 글자라서 숫자 계산을 할 수 없습니다.

```
a = 123
b = "123"
print(a + 1)    # 124
print(b + 1)    # TypeError! (글자 + 숫자)
```

```
▶ 124
▶ TypeError: can only concatenate str ... to str
```

왜 쓰나 진짜 숫자는 더하기·빼기·나누기가 되지만, 문자열 "123"은 계산하면 오류(TypeError)가 납니다. 이럴 땐 종류를 바꿔줘야 하는데, 그게 다음 시간의 "형변환"입니다(`int("123") → 123`).

자주 하는 실수

나눗셈은 딱 떨어져도 항상 실수(float)입니다: `10 / 2`는 5가 아니라 5.0. 정수 몫이 필요하다면 `//`를 쓰세요 (`10 // 2 → 5`).

STEP 3

산술 연산 완성하기

어제의 사칙연산에 거듭제곱과 우선순위를 더하다

어제 `+`, `-`, `*`, `/`, `//`, `%`를 봤습니다. 오늘은 거듭제곱 `**`을 더하고, 여러 연산이 섞였을 때의 "우선순위"를 확실히 잡습니다. 연산자는 결국 값으로 계산하는 기호이고, 데이터 분석은 이 계산의 연속입니다.

연산자 거듭제곱 `**` 제곱·세제곱

정의 왼쪽 수를 오른쪽 수만큼 거듭 곱합니다. `2**3`은 2를 세 번 곱한 8.

```
print(2 ** 3)    # 8    (2×2×2)
print(5 ** 2)    # 25   (넓이: 변×변)
```

```
▶ 8
▶ 25
```

왜 쓰나 정사각형 넓이(`변**2`), 부피, 지수 계산에 씁니다. 어제는 없던 새 연산자예요.

연산자 몫 `//` · 나머지 `%` 나눗셈의 두 조각

정의 `//`는 나눈 몫(정수), `%`는 나눈 나머지입니다.

```
print(7 // 2)    # 3   (몫)
print(7 % 2)     # 1   (나머지)
minutes = 500
print(minutes // 60) # 8   (시간)
print(minutes % 60)  # 20  (분)
```

```
▶ 3
▶ 1
▶ 8
▶ 20
```

왜 쓰나 "500분은 몇 시간 몇 분?" 같은 변환에 딱입니다. //로 시간(8), %로 남은 분(20). 짝수/홀수 판별(% 2)에도 자주 씁니다.

설비 적용

설비 예: 부품 47개를 상자당 6개씩, 남김없이 담을 상자 수 = $(47 + 6 - 1) // 6 = 8$ 개. "올림 나눗셈" 관용구입니다.

연산자 연산자 우선순위 & 괄호 무엇을 먼저 계산하나

정의 거듭제곱(**)이 가장 먼저, 그다음 *, /, //, %, 마지막이 +, -. 순서를 바꾸려면 괄호를 씁니다.

```
print(2 + 3 * 4)    # 14   (곱 먼저: 3*4=12, +2)
print((2 + 3) * 4)  # 20   (괄호 먼저: 5*4)
print(2 ** 3 + 1)   # 9    (거듭제곱 먼저: 8+1)
```

```
▶ 14
▶ 20
▶ 9
```

왜 쓰나 평균처럼 "먼저 더하고 나눠야" 하는 계산은 괄호가 필수입니다: $(80 + 90) / 2$. 헛갈리면 무조건 괄호로 순서를 못 박으세요.

연산자 복합 할당 += · 변수에 저장 계산 결과를 담기

정의 계산 결과를 변수에 저장하고, +=는 "자기 값에 더해 다시 저장"하는 줄임 표현입니다.

```
result = 3 + 5
print(result)    # 8
score = 100
score += 10      # score = score + 10
print(score)     # 110
```

```
▶ 8
▶ 110
```


왜 쓰나 어제 배운 재할당($x = x + 5$)을 한 번 더, 그리고 그 줄임형 +=를 익힙니다. 점수 누적·카운터 증가에 자주 씁니다 (-=, *= 도 같은 방식).

연산자 문자열의 + 와 * (복습·심화) 글자 이어붙이기·반복

정의 문자열에서 +는 이어붙이기, *는 반복. 숫자와 문자열을 +로 섞으면 오류입니다.

```
print("안녕" + "하세요")    # 안녕하세요
print("-" * 10)             # -----
print(3 + "5")              # TypeError: int + str
```

▶ 안녕하세요
▶ -----
▶ TypeError: unsupported operand ... 'int' and 'str'

왜 쓰나 "=" * 30 처럼 구분선을 그리거나 라벨을 조립할 때 유용합니다. 단, 숫자 3과 글자 "5"를 그냥 더하면 종류가 안 맞아 TypeError — str()로 맞춰야 합니다(어제 배운 형변환).

STEP 4

비교 연산자 — 판단의 시작

두 값을 견주어 참·거짓(bool)을 만든다

여기서부터가 오늘의 하이라이트입니다. 비교 연산자는 두 값을 견주어 참(True)/거짓(False)을 냅니다. 이 결과가 바로 앞에서 배운 bool 자료형이고, 설비가 "기준을 넘었는가?"를 판정하는 출발점입니다.

비교 == (같다) · != (다르다) 같은지/다른지

정의 두 값이 같으면 ==가 True, 다르면 !=가 True. (어제 배운 =는 "저장", ==는 "비교"임을 다시 기억!)

```
print(5 == 5)    # True
print(5 != 3)    # True
print(5 == 3)    # False
```

▶ True
▶ True
▶ False

왜 쓰나 "측정값이 목표값과 같은가?", "상태가 정상과 다른가?"를 판정합니다. 가장 많이 쓰는 비교예요.

자주 하는 실수

=와 ==를 헷갈리는 것이 초보자의 1순위 실수입니다. temp = 85는 "저장", temp == 85는 "85와 같니?"(참/거짓). 비교엔 반드시 등호 두 개.

비교 < > <= >= 크기 비교

정의 작다·크다·작거나 같다·크거나 같다. <=와 >=는 "같아도 참"입니다.

```
print(5 > 3)      # True
print(5 >= 5)     # True  (같아도 참)
print(1 > 10)     # False
```

- ▶ True
- ▶ True
- ▶ False

왜 쓰나 "온도 >= 85면 과열", "진동 > 5면 이상"처럼 임계값 판정의 핵심입니다. 설비 감시의 대부분이 이 크기 비교로 이뤄집니다.

개념 비교 결과는 항상 bool True 아니면 False

정의 모든 비교의 결과는 참(True) 또는 거짓(False), 즉 bool 자료형입니다.

```
result = 5 > 3
print(result)      # True
print(type(result)) # <class 'bool'>
```

- ▶ True
- ▶ <class 'bool'>

왜 쓰나 STEP 1의 bool과 여기가 연결됩니다. 비교가 bool을 "만들어내는" 공장인 셈이죠. 이 True/False를 변수에 담아 두고두고 쓸 수 있습니다.

비교 문자열도 비교할 수 있다 글자끼리 같은지

정의 문자열도 ==로 같은지 비교합니다. 단, 대소문자는 다른 글자로 봅니다.

```
print("정상" == "정상") # True
print("A" == "a")       # False  (대소문자 구분!)
```

- ▶ True
- ▶ False

왜 쓰나 상태 문자열("정상"/"경고")을 비교해 판정할 때 씁니다. 완전히 똑같은 글자일 때만 True — 띄어쓰기 하나, 대소문자 하나만 달라도 False입니다.

STEP 5

논리 연산자 — 조건 연기

and · or · not 으로 여러 판단을 하나로

비교 하나로는 "온도 90도 초과"까지만 못 봅니다. 실제 판단은 "온도가 정상 범위 안(70 이상 그리고 85 이하)"처럼 여러 조건이 엮입니다. 그 엮음을 담당하는 것이 논리 연산자 and · or · not입니다.

논리 and (그리고) 둘 다 참이어야 참

정의 양쪽이 모두 True일 때만 True. 하나라도 False면 False.

```
print(True and True)    # True
print(True and False)   # False
print(85 > 80 and 85 < 100) # True (80 초과 그리고 100 미만)
```

- ▶ True
- ▶ False
- ▶ True

왜 쓰나 "정상 범위 안에 있는가?"처럼 여러 기준을 동시에 만족해야 할 때 씁니다. 범위 판정(하한 그리고 상한)의 기본이에요.

논리 or (또는) 하나라도 참이면 참

정의 양쪽 중 하나라도 True면 True. 둘 다 False일 때만 False.

```
print(True or False)    # True
print(False or False)   # False
```

- ▶ True
- ▶ False

왜 쓰나 "온도 초과 또는 진동 초과 또는 압력 0 → 정비 필요"처럼, 여러 위험 신호 중 하나라도 걸리면 조치해야 할 때 씁니다.

설비 적용

설비 판정식: `need_fix = temp > 90 or vib > 5 or pressure == 0` — 셋 중 하나만 참이어도 정비 필요(True).

논리 not (아니다) 참·거짓 뒤집기

정의 True는 False로, False는 True로 뒤집습니다.

```
print(not True)          # False
print(not False)         # True
temp = 85
print(not (temp > 90))    # True   (90 초과가 아니다)
```

```
▶ False
▶ True
▶ True
```

왜 쓰나 "~가 아니다", "정상이 아니다"를 표현합니다. 조건을 반대로 뒤집어 읽어야 할 때 유용해요.

논리 논리 조합 & 우선순위 and가 or보다 먼저

정의 여러 논리를 엮을 때 and가 or보다 먼저 계산됩니다. 헷갈리면 괄호로 순서를 명확히.

```
print(True or False and False)    # True   (and 먼저: F and F=F, T or F=T)
print((True or False) and False)  # False  (괄호로 or 먼저)
```

```
▶ True
▶ False
```

왜 쓰나 산술에 우선순위가 있듯 논리에도 있습니다. 실무에서는 의미가 헷갈리지 않게 괄호로 묶어 쓰는 것을 권합니다.

개념

오늘의 마무리: 비교 6종(== != < > <= >=)과 논리 3종(and·or·not)이 "판단"의 전부입니다. 지금은 참·거짓을 만들어 print로 확인하는 단계이고, 다음 시간 "조건문(if)"에서 이 참·거짓으로 갈림길을 결정하게 됩니다. 오늘 만든 True/False가 곧 조건문의 연료입니다.

부록 A

자료형·연산자 치트시트

오늘 배운 것 한눈에 — 복습·시험용

시험이나 복습 전에 아래 두 표만 훑어도 오늘 배운 자료형(무엇을 담는가)과 연산자(무엇을 하는가)가 한 번에 정리됩니다.

A-1. 기본 자료형 4가지

자료형	무엇	예시	판별
int 정수	소수점 없는 수(개수·나이)	3 27 0	따옴표X·소수점X
float 실수	소수점 있는 수(온도·평균)	25.5 5.0	따옴표X·소수점O
str 문자열	따옴표로 감싼 글자(이름·상태)	"센서A" "3"	따옴표O
bool 불린	참·거짓(정상·비정상)	True False	True/False

A-2. 연산자 총정리

갈래	연산자	뜻·예시
산술	+ - * /	사칙연산(/ 는 결과가 실수 5.0)
산술	// % **	몫·나머지· 거듭제곱(2**3=8)
비교	== !=	같다· 다르다 → True/False
비교	< > <= >=	크기 비교(<=,>= 는 같아도 참)
논리	and or not	둘 다· 하나라도· 뒤집기

개념

우선순위 한 줄 요약: ** → * / // % → + - → 비교(< > == 등) → not → and → or. 헛갈리면 괄호로 못 박기.

부록 B

오류·함정 사전

오늘 코드에서 나오기 쉬운 함정 모음

오늘 코드에서 자주 걸려 넘어지는 지점만 모았습니다. 왼쪽처럼 쓰면 오른쪽으로 고치면 됩니다. 대부분 "숫자인 줄 알았는데 문자열"이라서 생깁니다.

X 이렇게 하면	무슨 일	✓ 바로잡기
<code>a = true</code>	NameError(참은 True)	<code>a = True</code>
<code>"123" + 1</code>	TypeError(글자+숫자)	<code>int("123") + 1</code>

✗ 이렇게 하면	무슨 일	✓ 바로잡기
<code>3 + "5"</code>	TypeError(숫자+글자)	<code>3 + int("5")</code>
<code>0.1 + 0.2 == 0.3</code>	False(실수 오차)	<code>round(0.1+0.2,1)==0.3</code>
<code>"A" == "a"</code>	False(대소문자 구분)	철자·대소문자 통일
<code>if temp = 85</code>	= 는 저장(비교 아님)	<code>temp == 85</code>
<code>10 / 2 → 5</code>	실제는 5.0(나눗셈=실수)	<code>10 // 2 → 5</code>

주의

공통 처방: 계산이 이상하면 **type()**으로 **종류부터 확인**하세요. 오늘 함정의 절반은 "숫자인 줄 알았는데 문자열"이라서 생깁니다. print(값)과 type(값)을 나란히 찍어보는 2단계 점검이 가장 빠른 해결책입니다.

부록 C

설비 실습 하이라이트 & 다음 시간

오늘 배운 연산으로 실제 설비 지표를 계산하다

연산자 교안의 실습들은 모두 설비 현장을 소재로 합니다. 오늘 배운 산술·비교·논리가 실제로 어떻게 쓰이는지, 대표 실습 몇 개를 코드로 정리했습니다.

불량률 · 가동률 (산술)

```
total = 500
defect = 23
print(defect / total * 100)    # 4.6   (불량률 %)
run_h = 21
all_h = 24
print(run_h / all_h * 100)    # 87.5 (가동률 %)

▶ 4.6
▶ 87.5
```

설비 상태 종합 판정 (산술 + 비교 + 논리)

```
t1 = 78; t2 = 80; t3 = 82
avg = (t1 + t2 + t3) / 3
print(avg)                # 80.0
temp_ok = 70 <= avg and avg <= 85
print(temp_ok)            # True
total = 500; defect = 20
print(temp_ok and (defect / total * 100 < 5)) # True (양호)
```

```
▶ 80.0
▶ True
▶ True
```

설비 적용

오늘 배운 세 갈래가 한 줄에 모두 들어갑니다: **산술**로 평균을 내고, **비교**로 정상 범위를 판정하고, **논리(and)**로 "온도 정상 그리고 불량률 5% 미만 → 양호"를 결정합니다. 이것이 오늘 배운 것의 실전 모습입니다.

다음 시간 예고

다음 주제	한 줄 미리보기
조건문 (if / elif / else)	오늘 만든 True/False로 "갈림길"을 결정 — 참이면 이 코드, 아니면 저 코드
입력 (input)	input()으로 사용자·현장에서 값을 받기(받은 값은 항상 문자열)
형변환 (int / float / str)	받은 문자열 "85"를 숫자 85로 바꾸기 — 오늘의 TypeError 함정 해결

연결

오늘 "문자열 \"123\"은 계산이 안 된다(TypeError)"는 함정을 만났죠. 다음 시간의 **형변환**이 바로 그 해결책입니다. 그리고 오늘 만든 참·거짓은 **조건문**의 연료가 됩니다. 오늘 배운 것이 다음 시간에 곧바로 이어집니다.

맺음말

오늘의 성취와 다음 한 걸음

종류 → 계산 → 판단, 한 바퀴 완성

오늘 하루로 값의 "종류"를 구분하고(자료형 int·float·str·bool), type()으로 확인하고, 산술로 계산하고(거듭제곱·우선순위까지), 비교·논리로 참·거짓을 만들어 "판단"하는 데까지 왔습니다. 어제의 변수(값에 이름)에서 시작해, 오늘 그 값을 다루는 도구를 손에 쥔 셈입니다. 이 참·거짓을 실제 "행동"으로 바꾸는 조건문이 다음 시간에 기다립니다.

개념

복습 루틴 추천: ① `04\_variable.py`의 값들에 `type()`을 붙여 네 자료형을 눈으로 확인 → ② `0.1 + 0.2`,
`"3" + "5"`, `10 / 2`를 직접 실행하며 "왜 이 결과인지" 한 문장으로 설명해 보기 → ③ `temp = 88`을
두고 `temp >= 85 and temp < 100`처럼 판정식을 직접 만들어 True/False를 예측한 뒤 실행. 예측과
결과가 맞는지 대조하는 습관이 다음 시간 조건문을 쉽게 만듭니다.

— 끝 —